Goedel-Code-Prover: Hierarchical Proof Search for Open State-of-the-Art Code Verification

Zenan Li[1,*], **Ziran Yang**[2,*], **Deyuan He**[3], **Haoyu Zhao**[3], **Andrew Zhao**[3], **Shange Tang**[2], **Kaiyu Yang**[4], **Aarti Gupta**[3], **Zhendong Su**[1], **Chi Jin**[2]
[1]ETH Zürich, [2]Princeton Language and Intelligence,
[3]Department of Computer Science, Princeton University, [4]Apodex
[*]Equal contribution
{zenan.li@inf.ethz.ch, zirany@princeton.edu, chij@princeton.edu}

arXiv:2603.19329v3 [cs.SE] 10 Aug 2026

Abstract

Large language models (LLMs) can generate plausible code but offer limited guarantees of correctness. Formally verifying that implementations satisfy specifications requires constructing machine-checkable proofs, a task that remains beyond current automation. We propose a hierarchical proof search framework for automated code verification in Lean 4 that decomposes complex verification goals into structurally simpler subgoals before attempting tactic-level proving. Central to our approach is a principled decomposition score that combines constructive justification with structural effectiveness. The same score serves as both the training reward and the inference-time ranking criterion, aligning optimization and deployment. We train *Göedel-Code-Prover-8B*, a single unified policy for both decomposition and completion, through supervised initialization followed by hybrid reinforcement learning, where a continuous decomposition reward supports planning exploration while supervised replay stabilizes proof generation. On three Lean-based code verification benchmarks comprising 427 tasks, our 8B-parameter model achieves a 62.0% prove success rate, a 2.6× improvement over the strongest baseline under the reported inference settings. We further observe consistent inference-time scaling: success rates improve monotonically with search iterations and sampling budget, while whole-proof baselines plateau within the evaluated budgets.

1 Introduction

Large language models (LLMs) have rapidly advanced code generation, achieving strong performance on programming benchmarks and increasingly serving as real-world coding assistants (Chen et al., 2021; Li et al., 2022; Alharbi & Alshayeb, 2026; Li et al., 2026). Yet while LLM-based coding agents routinely generate unit tests and reproduction scripts to validate their outputs, empirical studies show that such testing cannot certify the absence of subtle logical errors, boundary-case violations, or specification mismatches (Jesse et al., 2023; Liu et al., 2023; Al-Kaswan et al., 2026). As LLMs are deployed in safety-critical environments, bridging the gap between "code that appears to work" and "code that is provably correct" becomes increasingly urgent.

Formal verification offers a principled solution by expressing specifications (see Figure 1 for an example) in a formal language and constructing machine-checked proofs of correctness. Lean 4 (Moura & Ullrich, 2021), a modern interactive theorem prover, provides a powerful platform for such verification tasks, with a growing ecosystem of libraries (e.g., `Mathlib` (mathlib Community, 2020) and `CSLib` (Barrett et al., 2026)). Figure 1 illustrates a typical verification task: given a function `FindSingleNumber` that returns the unique element from a list, the specification encodes a precondition and a postcondition as Lean predicates, and the top-level theorem asserts that the implementation satisfies this specification. The `sorry` placeholder marks the proof obligation to be discharged. Even for this seemingly simple problem, constructing a complete proof requires discovering auxiliary lemmas about list

```
-- Precondition: every element appears 1 or 2 times; exactly one is unique
def FindSingleNumber_precond (nums : List Int) : Prop :=
  let numsCount := nums.map (fun x => nums.count x)
  numsCount.all (fun c => c = 1 ∨ c = 2) ∧ numsCount.count 1 = 1

-- Implementation: find the unique element that appears exactly once
def filterlist (x : Int) (nums : List Int) : List Int :=
  nums.filter (fun y => y == x)

def FindSingleNumber (nums : List Int)
    (h : FindSingleNumber_precond nums) : Int :=
  match nums.filter (fun x => (filterlist x nums).length == 1) with
  | x :: _ => x
  | []     => 0

-- Postcondition: result is the unique element; others appear twice
def FindSingleNumber_postcond (nums : List Int) (result : Int)
    (h : FindSingleNumber_precond nums) : Prop :=
  nums.length > 0 ∧ (filterlist result nums).length = 1 ∧
  ∀ (x : Int), x ∈ nums → x = result ∨ (filterlist x nums).length = 2

-- Verification goal (proof to be synthesized automatically)
theorem FindSingleNumber_spec (nums : List Int)
    (h : FindSingleNumber_precond nums) :
    FindSingleNumber_postcond nums (FindSingleNumber nums h) h := by
  sorry
```

Figure 1: A code verification task in Lean 4. **Blue**: specification (precondition and postcondition). **Green**: implementation. **Red**: verification goal (`sorry` marks the proof obligation). Full proof in Appendix E.

filtering, membership, and inductive reasoning over the input structure, none of which are suggested by the specification itself. Writing such proofs by hand remains labor-intensive and demands substantial expertise, limiting widespread adoption of formal verification.

Recent work has explored leveraging LLMs to automate proof synthesis in interactive theorem provers (ITPs) such as Lean 4 and Isabelle (Li et al., 2024; Yang et al., 2024a). State-of-the-art systems have achieved remarkable success in formal mathematics, reaching olympiad-level performance (Ren et al., 2025; Lin et al., 2025; Xin et al., 2025b; Hubert et al., 2025; Wang et al., 2025). These methods typically operate in a *decompose-and-prove* paradigm: the model first decomposes a complex theorem into simpler intermediate lemmas (or subgoals), then proves each lemma individually using tactic generation.

Despite this progress, these methods do not transfer directly to code verification due to two fundamental mismatches. The first is *ungrounded decomposition*: unlike mathematics, where proof reasoning pervades textbooks and online discussions, virtually no natural-language corpus describes how program specifications are proved (Myers et al., 2004; Ammann & Offutt, 2017). LLMs therefore lack the informal-to-formal bridge that enables effective decomposition in mathematics, and their proposed subgoals are frequently imprecise or no simpler than the original goal. The second is *compound domain shift*: each program introduces its own functions and invariants without library support (*concept proliferation*), and discharging subgoals requires program-specific tactics that differ substantially from those prevalent in mathematics (*tactic distribution shift*).

In this work, we propose a *hierarchical proof search* framework for automated code verification in Lean 4, in which the verification goal is recursively decomposed into a series of lemmas that are then proved iteratively (Figure 2). To achieve this, we train *Göedel-Code-Prover-8B*, a single unified policy that performs both decomposition and completion. Since the quality of decomposition is the primary determinant of success, we introduce a *decomposition score* as both the training reward and inference-time search criterion, evaluating candidates

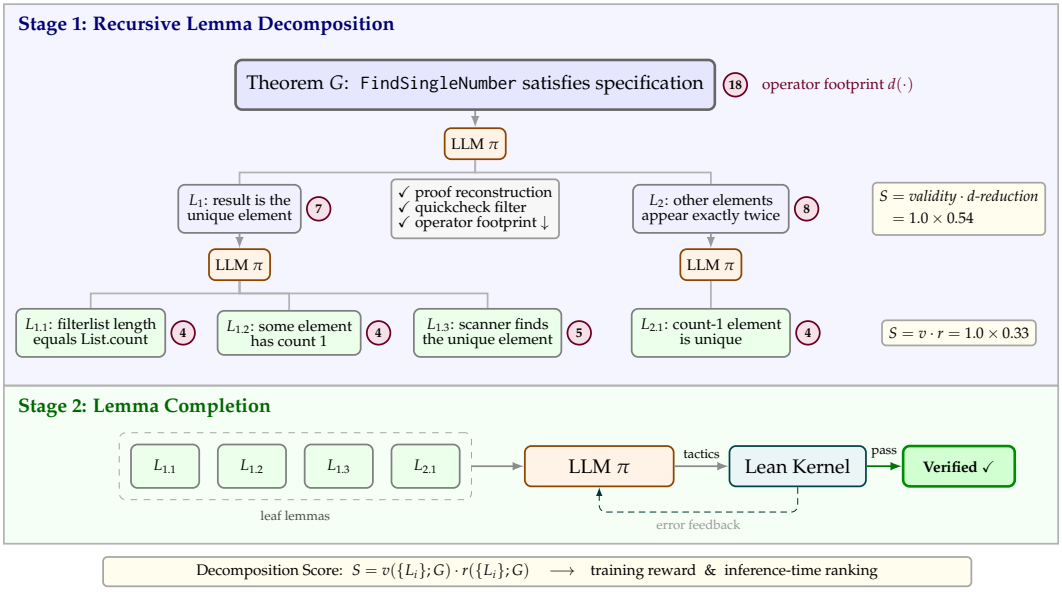

Figure 2: **Overview.** *Stage 1:* The goal is recursively decomposed into simpler sub-lemmas; the score S ranks candidates by validity and structural reduction in operator footprint $d(\cdot)$. *Stage 2:* Leaf lemmas are proved via iterative tactic generation with Lean 4 compiler feedback. The score serves as both training reward and inference-time ranking criterion.

along two axes: constructive justification (whether proposed subgoals provably entail the parent theorem and survive automated counterexample search via *quickcheck* (Claessen & Hughes, 2000; Lean Prover Community, 2024)), and structural simplicity (measured by a goal complexity metric). After supervised initialization, we apply hybrid reinforcement learning as a modest refinement: the continuous score optimizes decomposition while supervised replay stabilizes completion.

At inference time, the model first decomposes the verification goal into subgoals (*lemma decomposition*), then proves each via iterative tactic generation (*lemma completion*); the decomposition score ranks candidate decompositions, while the unified policy performs both stages.

We evaluate our framework on three Lean 4 code verification benchmarks: Verina (Ye et al., 2025), Clever (Thakur et al., 2025), and AlgoVeri (Zhao et al., 2026), comprising 427 tasks in total. Our 8B-parameter model achieves success rates of 68.8%, 54.0%, and 62.3% respectively, a 2.6× improvement over the strongest baseline and a 38.2-point absolute gain over the best neural prover under the reported inference settings. Verified proofs average 8–17 auxiliary lemmas and 130–167 lines of proof code across benchmarks, with the most complex reaching 680 lines, demonstrating that our framework can sustain deep structured reasoning over non-trivial verification tasks. We further observe consistent inference-time scaling: success rates improve monotonically with search iterations and sampling budget, indicating unsaturated scaling potential.

In summary, our contributions are: (1) a hierarchical proof search framework that decomposes complex verification goals into simpler subgoals before tactic-level proving; (2) a decomposition score combining constructive justification with structural simplicity, used as both training reward and inference-time search criterion; (3) a hybrid training pipeline that jointly trains decomposition and completion within a unified policy, with RL providing a modest but consistent refinement over supervised initialization; and (4) extensive evaluation showing that a purpose-trained 8B model outperforms the reported frontier-model and neural-prover baselines under their respective inference settings.

2 Lean-based Code Verification

We model automated code verification in Lean 4 using a Hoare-style specification framework. Concretely, we are given:

- a *program* C (a Lean definition or function),
- a *precondition* P and a *postcondition* Q, both expressed as Lean predicates.

The verification objective is to establish a Hoare-style triple

$$\{P\}\ C\ \{Q\},$$

which asserts that for any input x satisfying $P(x)$, the evaluation of $C(x)$ yields a result v such that $Q(x, v)$ holds. In the functional setting of Lean, C is typically a pure function, and the triple is encoded as a Lean proposition relating inputs and outputs of C. In our running example (Figure 1), $C =$ `FindSingleNumber`, P requires every element appears once or twice with exactly one unique element, and Q asserts the output is that unique element.

Formally, the input to the verification task is a tuple (C, P, Q) expressed in Lean, and the goal is to construct a machine-checkable proof term τ such that the Lean kernel accepts τ as a valid derivation of the verification goal G:

$$G\ :\ \forall x, P(x) \to Q(x, C(x)).$$

We assume that the program C and its associated predicates P and Q are already formalized in Lean; translating informal specifications into Lean is out of scope. The core challenge is therefore the synthesis of the proof term τ, which often involves quantifier elimination, complex induction, invariant generation, and the formulation of auxiliary assertions.

When we employ an LLM to automate this synthesis, the task reduces to interactive tactic generation: the LLM observes a *goal state* (local hypotheses plus target proposition), issues a tactic command, and the Lean kernel checks it, updates the proof state, and generates new subgoals. This loop continues until all goals are discharged or the attempt is abandoned.

We choose Lean 4 over SMT-based verifiers (e.g., Dafny, Verus) because its interactive proof protocol exposes explicit goal states and fine-grained compiler diagnostics, enabling iterative LLM-driven search; a detailed comparison is provided in Appendix C. As discussed in Section 1, automated code verification poses three key challenges beyond formal mathematics—ungrounded decomposition, concept proliferation, and tactic distribution shift—which together create a compound search problem with an exceptionally fragile search space (see Appendix D for an extended analysis).

3 Hierarchical Verification Framework

Our framework centers on a *decomposition score* that quantifies candidate lemma decompositions and serves consistently as both the training reward and the inference-time ranking criterion. We first define the score (section 3.1) and then describe the unified training and inference pipeline (section 3.2).

3.1 Score for Lemma Decomposition

Lemma decomposition is a critical bottleneck: a good decomposition turns a global correctness theorem into provable subgoals, whereas a poor one produces invalid or equally intractable obligations. We seek a *computable* score that correlates with downstream provability, is cheap to evaluate online, and serves consistently as both the training reward and the inference-time ranking criterion. We define the score along two complementary axes: *(i) constructive justification* and *(ii) decomposition effectiveness*.

Constructive justification. A lemma decomposition is only meaningful if the proposed lemmas $(L_1, \ldots, L_k)$ logically entail the parent theorem (G). Specifically, a candidate decomposition must provide a proof reconstruction: a proof term π_{parent}, produced by the LLM as part of its decomposition output, such that Lean can verify the implication $(L_1 \wedge \cdots \wedge L_k) \Rightarrow G$.

We require that π_{parent} explicitly invokes each proposed lemma, ensuring the decomposition is not merely a syntactic list but a constructively justified reduction of the original goal.

However, constructive justification alone does not guarantee the validity of the subgoals: a false lemma (e.g., `l = []` $\wedge$ `l.length > 0`) can technically prove the parent theorem but shifts the burden to an impossible sub-task. To filter out such cases, we integrate *quickcheck* as a semantic gate: it randomly samples concrete inputs and checks whether each proposed lemma holds. If a counterexample is found for any L_i, the entire decomposition is discarded before expensive proof search is attempted.

Decomposition effectiveness. A decomposition is effective if it reduces the difficulty of the original verification task. In Hoare-triple terms, consider splitting $\{P\}\, C\, \{Q\}$ into $\{P\}\, C_1\, \{R\}$ and $\{R\}\, C_2\, \{Q\}$, where $C_1; C_2$ is a partition of the program C and R is an intermediate assertion characterizing the state between the two fragments. Such a decomposition is productive when each resulting sub-goal is strictly simpler than the original, either by simplifying the logical assertions (i.e., R simpler than P and Q) or the programs (i.e., C_1, C_2 simpler than C).

To quantify this notion of simplicity, we observe that in Lean's abstract syntax tree (AST), both assertions and programs are uniformly expressed through operators: logical operators (connectives and quantifiers) and program operators (arithmetic, bitwise, or data-structure constructors). We define the structural difficulty $d(G)$ of a verification goal G as its *operator footprint*: the total number of logical and domain-specific operator occurrences in its AST. This metric captures reasoning cost because each operator corresponds to a specific class of proof obligations: reducing logical operators simplifies propositional reasoning, and reducing computational operators simplifies reasoning about program behavior.

We illustrate this with the running example (Figure 1). The goal `FindSingleNumber_spec` combines specification operators ($\forall$, $\rightarrow$, $\wedge$, $\vee$, $\in$, $=$) with program operators (`filterlist`, `List.count`, `List.map`, `List.length`), yielding $d(G) = 18$. A decomposition splits G into L_1: "`filterlist` on the unique element returns a singleton list" ($d(L_1) = 7$) and L_2: "every other element appears in `filterlist` exactly twice" ($d(L_2) = 8$). Each sub-lemma isolates a single property of `filterlist` with a smaller footprint, confirming a positive structural reduction.

3.2 Unified Policy Learning and Inference

We implement two custom Lean tactics to compute the score: `operatorcount` for the operator footprint and `quickcheck` for automated counterexample search. The score S combines a binary validity gate with a continuous structural reduction ratio. A candidate decomposition is valid only if (i) the proposed lemmas collectively discharge G via proof reconstruction, and (ii) every lemma survives quickcheck without counterexamples. We encode this as

$$v(L_1, \ldots, L_k; G) = \mathbb{1}_{\text{proof}}(L_1, \ldots, L_k; G) \cdot \prod_{i=1}^{k} \mathbb{1}_{\text{qc}}(L_i),$$

where $\mathbb{1}_{\text{proof}}(L_1, \ldots, L_k; G) = 1$ if proof reconstruction succeeds and 0 otherwise, and $\mathbb{1}_{\text{qc}}(L_i) = 1$ if no counterexample is found for L_i and 0 otherwise.

Next, we quantify how much simpler the sub-goals are relative to the original. We aggregate the individual footprints $d(L_i)$ via LogSumExp, which approximates $\max_i d(L_i)$ while remaining sensitive to reductions in any individual subgoal:

$$\bar{d}(L_1, \ldots, L_k) = T \cdot \log \sum_{i=1}^{k} \exp\left(d(L_i)/T\right),$$

where T controls the smoothness. The structural reduction ratio is then

$$r(L_1, \ldots, L_k; G) = \max(1 - \frac{\bar{d}(L_1, \ldots, L_k)}{d(G)}, 0).$$

The final decomposition score is $S = r(L_1, \ldots, L_k; G) \cdot v(L_1, \ldots, L_k; G)$. Because this score is used identically during training (as the reward) and inference (as the ranking criterion),

policy learning and tree-search control are optimized against exactly the same objective. For clarity, the diagnostic plots in Figures 5, 10, 11, and 12 retain the unnormalized score prior to its conversion into S (their axes are labeled "Score"). This raw quantity is oriented as residual difficulty—lower values indicate simpler decompositions—and should not be confused with the normalized, higher-is-better score S defined above.

3.2.1 *Training Pipeline*

We train the same autoregressive policy for both planning and proving in two stages: (1) supervised initialization with scaffolded data; (2) hybrid reinforcement learning that optimizes decomposition while stabilizing completion.

Supervised fine-tuning (SFT). Using frontier models (GPT-5.2 and Gemini-3-Flash), we generate scaffolded trajectories for both lemma decomposition and lemma completion. This process yields 281K decomposition input–output pairs and 151K completion pairs. We merge the two types of trajectories and fine-tune a Qwen-3-8B backbone to obtain an initial policy π_0. At this stage, the model already supports both decomposition and completion behaviors under the same prompting scaffolding.

Hybrid reinforcement learning (RL). A central challenge in jointly optimizing decomposition and completion is reward mismatch: decomposition receives a dense, continuous score in $[0, 1]$, while completion yields only a sparse binary signal (proof accepted or rejected). Naively merging these objectives causes decomposition gradients to dominate and completion proficiency to stagnate.

We resolve this by decoupling the learning objectives while maintaining a shared parameter space. Decomposition is optimized via group relative policy optimization (GRPO), utilizing the structural score S to drive exploration and curriculum expansion. Completion is stabilized through supervised replay of high-quality proof trajectories (generated by either the policy itself or frontier models), avoiding reliance on brittle binary signals. Algorithm 1 (Appendix F) summarizes the full loop. For completion, we adopt a *policy-first* strategy: π attempts each lemma first, and a frontier model is invoked only when π fails within m attempts, ensuring that π is exposed to its own successful completions for replay. Additionally, successfully decomposed lemmas are fed back into the problem set, providing an ever-expanding curriculum of progressively simpler subgoals.

3.2.2 *Inference Pipeline*

Inference proceeds in two sequential stages: the decomposition stage runs first, iteratively breaking down the original goal until a budget is exhausted, after which the completion stage takes over and attempts to prove each remaining leaf lemma. We use the term *iteration* to refer to a single decomposition or completion step within its respective stage.

Lemma Decomposition. At each decomposition iteration, the policy selects the open goal with the highest operator footprint as the decomposition target, concentrating search effort on the most complex remaining obligation. The policy then rolls out a decomposition for the selected goal; the result is verified via proof reconstruction and quickcheck, and discarded if either check fails. Upon acceptance, the original goal is replaced by its sub-lemmas in the goal set, and operator footprints are computed for the new goals. This process repeats until the decomposition budget is exhausted or the open-lemma cap is reached.

Lemma Completion. After the decomposition stage concludes, the remaining open lemmas are passed to the completion stage. At each completion iteration, the policy generates a candidate proof for each unclosed lemma and submits it to the Lean compiler. If compilation fails, the error diagnostics are fed back to the policy, which revises the proof accordingly. This generate-and-refine loop repeats until all proofs are accepted or the budget is exhausted.

Algorithm 2 (Appendix F) summarizes the full inference pipeline. Beyond increasing the per-run budget, performance can be further improved by launching k independent runs in parallel and accepting the first successful proof (pass@k).

4 Experiments

4.1 Experimental Setup

Benchmarks. We evaluate on three Lean 4 code verification benchmarks. Verina (Ye et al., 2025) contains 189 tasks from introductory programming exercises, covering code generation, specification synthesis, and proof generation. Clever (Thakur et al., 2025) provides 161 problems with formal specifications designed to avoid test-case leakage and trivial solutions. AlgoVeri (Zhao et al., 2026) comprises 77 classical algorithms with identical functional contracts across Dafny, Verus, and Lean; we adopt its Lean subset. Among these, Verina ships complete programs with preconditions, implementations, and postconditions, forming a self-contained evaluation suite. Clever and AlgoVeri only release specifications without reference implementations; we therefore prompt GPT-5.2 to generate the code and manually verify the results. Together, the three benchmarks yield 427 verification tasks spanning straightforward functional correctness to deep algorithmic reasoning.

Baselines. We compare against two categories of baselines. (1) *Frontier reasoning models*: Claude-Opus-4.6, Gemini-3-Flash, GPT-5.2-Pro, GPT-5.3-Codex, and DeepSeek-V3.2-Speciale, each prompted to produce a complete proof in a single pass. (2) *Neural provers*: Kimina-Prover-72B (Wang et al., 2025), DeepSeek-Prover-V2-671B (Ren et al., 2025), Goedel-Prover-V2-32B (Lin et al., 2025), and BFS-Prover-V2-32B (Xin et al., 2025b), which are trained on large-scale formal proof corpora; among them, BFS-Prover employs best-first tree search over tactic sequences, while the others generate whole proofs. All baselines use default inference configurations and report pass@128. These comparisons are not compute-matched: our method performs iterative hierarchical search, whereas most baselines sample independent whole proofs or tactic sequences. We therefore report outcomes under the stated inference settings without attributing differences to model size alone; Appendix H.4 details the available budgets.

Implementation details. We fine-tune a Qwen-3-8B backbone (Yang et al., 2025) following our training pipeline; we refer to the resulting policy as *Göedel-Code-Prover-8B*. SFT uses 281K decomposition and 151K completion trajectories, followed by hybrid RL. At inference, the decomposition stage runs up to 128 iterations or until adding another decomposition would exceed 32 open lemmas; the completion stage refines each lemma for up to 128 iterations. Additional training hyperparameters, prompt templates, and inference configurations are provided in Appendix G.

Training data and decontamination. The 5K training problems are title/description-disjoint from all evaluation tasks. Audits on boilerplate-stripped Lean code find no shared substring of at least 200 characters between any benchmark and training implementation; an embedding audit flags only 2/427 pairs, both from Verina, and manual inspection finds that neither admits proof transfer. Threshold-level results and the flagged pairs are reported in Appendix H.2.

Metrics. We report the *prove success rate*: the fraction of problems for which a complete, Lean-verified proof is produced within the inference budget. To ensure soundness, we only admit the standard axioms `propext`, `Classical.choice`, and `Quot.sound`; proofs invoking `Lean.ofReduceBool` or `Lean.trustCompiler` are rejected, as these bypass the kernel's deductive checking and could mask unsound reasoning.

4.2 Overall Effectiveness

As shown in Figure 3, under the reported inference settings our framework achieves prove success rates of 68.8%, 54.0%, and 62.3% on Verina, Clever, and AlgoVeri, respectively, yielding an overall rate of 62.0% across all 427 tasks. In addition, quickcheck disproves 23, 10, and 14 problems on Verina, Clever, and AlgoVeri by producing concrete counterexamples.[1] Among frontier reasoning models, the strongest (GPT-5.3-Codex) reaches only 23.6% on

[1] Quickcheck is also applied directly to the top-level goal. When a counterexample is found, it is provided to the LLM, which then uses it as evidence to disprove the problem.

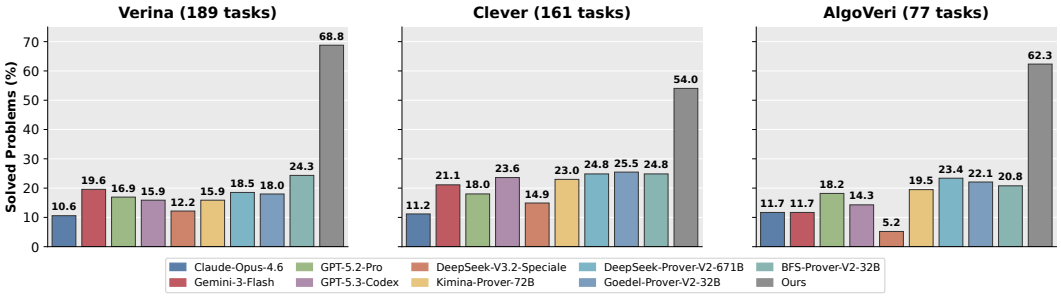

Figure 3: Percentage of solved problems by baselines and our framework across three benchmarks. Baselines are evaluated with parallel generation under a Pass@128 budget; our method operates under a search-based inference setting using Göedel-Code-Prover-8B. Our framework outperforms all baselines by a substantial margin across every benchmark.

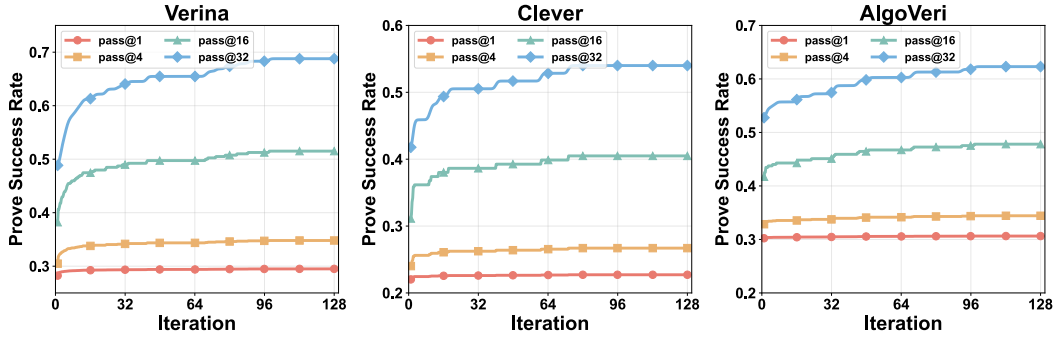

Figure 4: Prove success rate vs. completion iterations under different pass@k budgets.

Clever and below 20% on Verina and AlgoVeri, confirming that single-pass proof generation is insufficient for non-trivial verification. Among neural provers, BFS-Prover-V2-32B achieves the best overall score of 23.8%, while our framework is 38.2 percentage points higher under the reported, non-compute-matched settings.

On average, each verified problem requires 8 to 17 auxiliary lemmas and 130 to 167 lines of proof code across benchmarks, with the longest proofs reaching 680, 559, and 534 lines on Verina, Clever, and AlgoVeri (detailed statistics in Table 7, Appendix H).

4.3 Inference-Time Scaling

Our framework exposes two orthogonal scaling axes: the *per-run search budget* (number of iterations within each decomposition or completion stage) and the *parallel sampling budget* (pass@k, where k independent runs are launched and the first successful proof is accepted). As shown in Figure 8 (Appendix H.6), the residual operator-footprint ratio decreases steadily as iterations progress, indicating that the framework progressively identifies more effective goal simplifications with additional compute. Increasing the pass@k budget further lowers this ratio across all three benchmarks, confirming that broader parallel search at the decomposition stage produces structurally simpler sub-goals.

Figure 4 shows that the prove success rate grows steadily with both more completion iterations and larger pass@k, demonstrating clear inference-time scaling across all benchmarks. The persistent gap between pass@1 and pass@32 suggests that the framework benefits substantially from parallel search and has not yet saturated its scaling potential within the evaluated budget. By contrast, the pass@k curves of baseline verifiers plateau rapidly (see Appendix H.7), indicating that brute-force sampling without hierarchical decomposition yields diminishing returns.

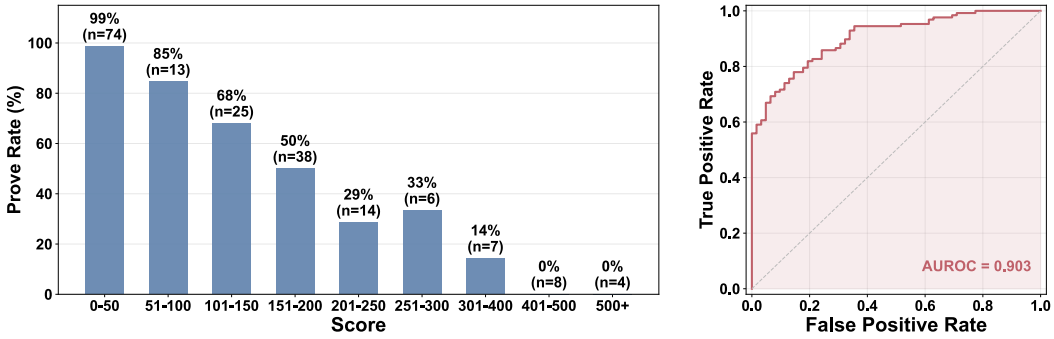

Figure 5: Unnormalized decomposition score vs. prove rate on Verina. Lower raw scores indicate simpler decompositions and separate instances proved within the evaluated budget from those left unsolved (AUROC = 0.903).

Table 1: Decomposition failure rates. Proof Failed: % of iterations where proof reconstruction rejects the decomposition. QC Failed: % of runs where at least one lemma fails quickcheck.

Dataset	Proof Failed	QC Failed
Verina	59.4	46.4
Clever	44.9	31.8
AlgoVeri	52.8	32.6

Table 2: Ablation study on each component. We progressively replace each component with our trained model to isolate its contribution.

Decomposition	**Completion**	Verina
—	Gemini-3-Flash	19.6
GPT-5.2-Pro	Gemini-3-Flash	54.4
Ours	Gemini-3-Flash	58.2
GPT-5.2-Pro	Ours	59.2
Ours	Ours	**68.8**

4.4 Decomposition Analysis

Table 1 quantifies how each criterion filters out unproductive decompositions. Among 32 parallel runs per problem, 31.8%–46.4% are discarded entirely because at least one proposed lemma fails quickcheck (QC Failed). Separately, proof reconstruction rejects 44.9%–59.4% of individual decomposition iterations (Proof Failed), confirming that constructing a logically sound reduction remains a major bottleneck. Because these rates use different denominators (runs versus iterations), they should not be added directly; together, they characterize complementary failure modes filtered by quickcheck and proof reconstruction.

Figure 5 further shows that the unnormalized decomposition score is a strong inverse predictor of downstream provability: on Verina, lower raw scores correlate with higher prove rates, achieving an AUROC of 0.903. Because the plotted quantity is the pre-normalization form of S, this result supports using the normalized, higher-is-better S to rank candidate decompositions during training and inference. The same inverse association appears on our training data (Appendix H.8).

4.5 Ablation Study

Table 2 isolates the contributions of our trained decomposition and completion modules by swapping each with baseline alternatives on Verina. Without decomposition, directly prompting Gemini-3-Flash for whole-proof generation achieves only 19.6%, confirming that hierarchical decomposition is essential. Adding GPT-5.2-Pro decomposition raises the rate to 54.4%; replacing it with our trained decomposer yields 58.2%, showing the value of domain-specific decomposition training. Similarly, replacing Gemini-3-Flash completion with our trained completer (GPT-5.2-Pro + Ours: 59.2%) shows that domain-specific completion training is at least as valuable as improved decomposition. Combining both trained components achieves 68.8%, surpassing every mixed configuration and confirming

Table 3: Matched-budget comparison of the SFT-only policy and the policy after RL on Verina. Entries are prove success rates (%).

Model	Pass@1	Pass@10	Pass@20	Pass@32
SFT only	26.9	44.9	53.9	66.1
SFT + RL	29.1	46.0	57.1	68.8
Difference	+2.2	+1.1	+3.2	+2.7

synergistic gains from joint training: the decomposer and completer co-adapt to each other's strengths. Because these module swaps compare models trained with both SFT and RL, they do not isolate the RL contribution. Table 3 therefore compares the same hierarchical policy before and after RL at matched inference budgets. RL adds a smaller but consistent 1.1–3.2 percentage points; the bulk of performance comes from hierarchical search and supervised training, and the single training run does not establish statistical significance. Separately, applying our framework to an off-the-shelf GPT-OSS-120B model raises Verina success from 20.1% with whole-proof generation to 44.9% with hierarchical search (Appendix H.1), showing that the structural benefit is not restricted to the frontier models used for data collection.

We also visualize how training shifts the unnormalized decomposition-score distribution in Appendix H.9: after training, raw scores shift toward lower values across all benchmarks, corresponding to simpler decompositions and higher normalized scores.

5 Related work

LLM-based theorem proving has advanced in mathematics, from tactic-level generation (Polu & Sutskever, 2020) and retrieval-augmented proving (Yang et al., 2024b) to large-scale neural provers (Xin et al., 2024; Wang et al., 2025; Lin et al., 2025) and RL-driven hierarchical decomposition (Hubert et al., 2025; Dong et al., 2024). These advances do not transfer directly to code verification because of concept proliferation and tactic distribution shift (Appendix D). For code verification, LLMs have generated SMT annotations (Kamath et al., 2023; Pei et al., 2023), produced or retrieved flat proofs (First et al., 2023; Thompson et al., 2024), and driven tactic-level Isabelle search with symbolic repair (He et al., 2026). The most closely related system, Quarry, ranks Rocq decompositions by predicted hammer solvability (Zhang et al., 2026); we instead train a unified Lean 4 policy using a code-specific decomposition score shared by training and inference. Learned compositional abstractions have also been studied for inductive program synthesis (Ellis et al., 2020), but not for decomposing ITP proof obligations. Our framework introduces a hierarchical planning layer with a continuous decomposition score, addressing the reward sparsity that limits prior RL approaches (Xin et al., 2025b; Lightman et al., 2024). A detailed discussion is provided in Appendix B.

6 Conclusion

We presented a hierarchical proof search framework for code verification in Lean 4, centered on a decomposition score that serves as both the training reward and the inference-time selection criterion. On three benchmarks, our 8B-parameter model achieves a 62.0% overall prove success rate, a 2.6× improvement over the strongest baseline under the reported inference settings. Its larger iterative inference budget is not compute-matched to parallel single-pass baselines; despite unsaturated scaling, reducing inference cost remains important. Resource constraints limited RL to one run, precluding across-seed variance and significance estimates. We also lack comparable token, API-cost, and GPU-hour accounting and per-task cross-system AlgoVeri outcomes. Further limitations are pre-formalized programs, syntactic operator counting, and single-procedure scope; informal specifications, semantic metrics, and interprocedural reasoning remain future work.

Acknowledgments

We thank the anonymous reviewers for their valuable comments. This work benefited from joint discussions and iterative refinement by all authors. Aarti Gupta acknowledges support from a Princeton SEAS Innovation Research Grant. Chi Jin acknowledges support from NSF grants OAC-2411299 and IIS-2239297 and the Princeton AI Lab Seed Fund.

References

Tudor Achim, Alex Best, Alberto Bietti, Kevin Der, Mathïs Fédérico, Sergei Gukov, Daniel Halpern-Leistner, Kirsten Henningsgard, Yury Kudryashov, Alexander Meiburg, et al. Aristotle: Imo-level automated theorem proving. *arXiv preprint arXiv:2510.01346*, 2025.

Wasi Uddin Ahmad, Aleksander Ficek, Mehrzad Samadi, Jocelyn Huang, Vahid Noroozi, Somshubra Majumdar, and Boris Ginsburg. Opencodeinstruct: A large-scale instruction tuning dataset for code llms. *arXiv preprint arXiv:2504.04030*, 2025.

Ali Al-Kaswan, Claudio Spiess, Prem Devanbu, Arie van Deursen, and Maliheh Izadi. Model see, model do? exposure-aware evaluation of bug-vs-fix preference in code llms. *arXiv preprint arXiv:2601.10496*, 2026.

Maha Alharbi and Mohammad Alshayeb. Automatic code generation techniques: A systematic literature review. *Automated Software Engineering*, 33(1):4, 2026.

Paul Ammann and Jeff Offutt. *Introduction to software testing*. Cambridge University Press, 2 edition, 2017. ISBN 9781107172012. doi: 10.1017/9781316771273.

Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, et al. cvc5: A versatile and industrial-strength smt solver. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pp. 415–442. Springer, 2022.

Clark Barrett, Swarat Chaudhuri, Fabrizio Montesi, Jim Grundy, Pushmeet Kohli, Leonardo de Moura, Alexandre Rademaker, and Sorrachai Yingchareonthawornchai. CSLib: The lean computer science library. *arXiv preprint arXiv:2602.04846*, 2026.

Jiangjie Chen, Wenxiang Chen, Jiacheng Du, Jinyi Hu, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Wenlei Shi, et al. Seed-prover 1.5: Mastering undergraduate-level theorem proving via learning from experience. *arXiv preprint arXiv:2512.17260*, 2025.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, pp. 268–279. ACM, 2000. ISBN 9781581132021. doi: 10.1145/351240.351266. URL https://dl.acm.org/doi/10.1145/351240.351266.

Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pp. 337–340. Springer, 2008.

Kefan Dong, Arvind Mahankali, and Tengyu Ma. Formal theorem proving by rewarding LLMs to decompose proofs hierarchically. *arXiv preprint arXiv:2411.01829*, 2024.

Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sable-Meyer, Luc Cary, Lucas Morales, Luke Hewitt, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: Growing generalizable, interpretable knowledge with wake–sleep Bayesian program learning. *arXiv preprint arXiv:2006.08381*, 2020.

Emily First, Markus N Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 1229–1241, 2023.

Baoding He, Zenan Li, Wei Sun, Yuan Yao, Taolue Chen, Xiaoxing Ma, and Zhendong Su. Neuro-symbolic proof generation for scaling systems software verification. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pp. 2533–2550. USENIX Association, 2026.

Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, pp. 607–613, 2025.

Kevin Jesse, Toufique Ahmed, Premkumar T Devanbu, and Emily Morgan. Large language models and simple, stupid bugs. *arXiv preprint arXiv:2303.11455*, 2023.

Albert Q Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. *arXiv preprint arXiv:2210.12283*, 2022.

Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.

Guillaume Lample, Timothée Lacroix, Marie-Anne Lachaux, Aurelien Rodriguez, Aitor Roziere, Thibaut Lavril, Thomas Scialom, and Gabriel Synnaeve. HyperTree proof search for neural theorem proving. In *Advances in Neural Information Processing Systems*, 2022.

Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying rust programs using linear ghost types. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1): 286–315, 2023.

Lean Prover Community. plausible. https://github.com/leanprover-community/plausible, 2024. GitHub repository.

Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. Deduplicating training data makes language models better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pp. 8424–8445, 2022.

K Rustan M Leino. Dafny: An automatic program verifier for functional correctness. In *International conference on logic for programming artificial intelligence and reasoning*, pp. 348–370. Springer, 2010.

Caihua Li, Lianghong Guo, Yanlin Wang, Daya Guo, Wei Tao, Zhenyu Shan, Mingwei Liu, Jiachi Chen, Haoyu Song, Duyu Tang, et al. Advances and frontiers of llm-based issue resolution in software engineering: A comprehensive survey. *arXiv preprint arXiv:2601.11655*, 2026.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.

Zhaoyu Li, Jialiang Sun, Logan Murphy, Qidong Su, Zenan Li, Xian Zhang, Kaiyu Yang, and Xujie Si. A survey on deep learning for theorem proving. *arXiv preprint arXiv:2404.09939*, 2024.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*, 2024.

Yong Lin, Shange Tang, Bohan Lyu, Ziran Yang, Jui-Hui Chung, Haoyu Zhao, Lai Jiang, Yihan Geng, Jiawei Ge, Jingruo Sun, et al. Goedel-prover-v2: Scaling formal theorem proving with scaffolded data synthesis and self-correction. *arXiv preprint arXiv:2508.03613*, 2025. doi: 10.48550/arXiv.2508.03613. URL https://arxiv.org/abs/2508.03613.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023.

The mathlib Community. The lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP 2020, pp. 367–381, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450370974. doi: 10.1145/3372885.3373824. URL https://doi.org/10.1145/3372885.3373824.

Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. Ray: A distributed framework for emerging AI applications. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*, pp. 561–577, 2018.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, pp. 625–635. Springer, 2021.

Glenford J Myers, Tom Badgett, Todd M Thomas, and Corey Sandler. *The art of software testing*. John Wiley & Sons, 2 edition, 2004. ISBN 9780471469124.

Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. Can large language models reason about program invariants? In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 27496–27520. PMLR, 2023. URL https://proceedings.mlr.press/v202/pei23a.html.

Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.

ZZ Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, et al. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. doi: 10.48550/arXiv.2402.03300. URL https://arxiv.org/abs/2402.03300.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.

Amitayush Thakur, Jasper Lee, George Tsoukalas, Meghana Sistla, Matthew Zhao, Stefan Zetzsche, Greg Durrett, Yisong Yue, and Swarat Chaudhuri. Clever: A curated benchmark for formally verified code generation. *arXiv preprint arXiv:2505.13938*, 2025.

Kyle Thompson, Nuno Saavedra, Pedro Carrott, Kevin Fisher, Alex Sanchez-Stern, Yuriy Brun, João F Ferreira, Sorin Lerner, and Emily First. Rango: Adaptive retrieval-augmented proving for automated software verification. *arXiv preprint arXiv:2412.14063*, 2024.

Haiming Wang, Mert Unsal, Xiaohan Lin, Mantas Baksys, Junqi Liu, Marco Dos Santos, Flood Sung, Marina Vinyes, Zhenzhe Ying, Zekai Zhu, et al. Kimina-prover preview: Towards large formal reasoning models with reinforcement learning. *arXiv preprint arXiv:2504.11354*, 2025.

Yunhui Xia, Wei Shen, Yan Wang, Jason Klein Liu, Huifeng Sun, Siyue Wu, Jian Hu, and Xiaolong Xu. Leetcodedataset: A temporal dataset for robust evaluation and efficient training of code llms. *arXiv preprint arXiv:2504.14655*, 2025. doi: 10.48550/arXiv.2504.14655. URL https://arxiv.org/abs/2504.14655.

Huajian Xin, Daya Guo, Zhihong Shao, Zhizhou Ren, Qihao Zhu, Bo Liu, Chong Ruan, Wenda Li, and Xiaodan Liang. DeepSeek-Prover: Advancing theorem proving in LLMs through large-scale synthetic data. *arXiv preprint arXiv:2405.14333*, 2024.

Ran Xin, Chenguang Xi, Jie Yang, Feng Chen, Hang Wu, Xia Xiao, Yifan Sun, Shen Zheng, and Ming Ding. Bfs-prover: Scalable best-first tree search for llm-based automatic theorem proving. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 32588–32599, 2025a.

Ran Xin, Zeyu Zheng, Yanchen Nie, Kun Yuan, and Xia Xiao. Scaling up multi-turn off-policy rl and multi-agent tree search for llm step-provers. *arXiv preprint arXiv:2509.06493*, 2025b.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Kaiyu Yang, Gabriel Poesia, Jingxuan He, Wenda Li, Kristin Lauter, Swarat Chaudhuri, and Dawn Song. Formal mathematical reasoning: A new frontier in ai. *arXiv preprint arXiv:2412.16075*, 2024a.

Kaiyu Yang, Aidan M Swope, Alex Gu, Rahul Chalapathy, Peiyang Song, Shixing Yu, Maruan Al-Shedivat, Jing Lei, and Eric P Xing. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, 2024b.

Zhe Ye, Zhengxu Yan, Jingxuan He, Timothe Kasriel, Kaiyu Yang, and Dawn Song. Verina: Benchmarking verifiable code generation. *arXiv preprint arXiv:2505.23135*, 2025.

Ning Zhang, Nongyu Di, Zenan Li, Yuan Yao, and Xiaoxing Ma. Planning to hammer: Difficulty-aware decomposition for automating Rocq proofs. *Proceedings of the ACM on Programming Languages*, 2026. OOPSLA 2026.

Haoyu Zhao, Ziran Yang, Jiawei Li, Deyuan He, Zenan Li, Chi Jin, Venugopal V Veeravalli, Aarti Gupta, and Sanjeev Arora. Algoveri: An aligned benchmark for verified code generation on classical algorithms. *arXiv preprint arXiv:2602.09464*, 2026.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi Feng, and Yongqiang Ma. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand, 2024. Association for Computational Linguistics. URL http://arxiv.org/abs/2403.13372.

A Disclosure of LLM Usage

As required by the COLM 2026 call for papers, we disclose all uses of large language models in preparing this work. In the research pipeline, GPT-5.2 and Gemini-3-Flash were used to generate Lean formalizations and training trajectories as described in Section 3 and Appendix G; GPT-OSS-120B was used in the open-model experiment, and `text-embedding-3-large` was used in the decontamination audit. LLM-assisted coding tools (Cursor and Claude Code) were used during software development, including experiment scripts and data-processing pipelines. ChatGPT and Gemini were additionally used for language editing and for drafting camera-ready revisions from author-provided technical content. The authors designed the research and experiments, verified generated formalizations and proofs using Lean and the checks described in the paper, analyzed the results, and take full responsibility for all final scientific claims.

B Extended Related Work

LLM-based theorem proving and the math-code gap. Language models have achieved remarkable progress in formal theorem proving, predominantly in mathematics. Starting from tactic-level generation (Polu & Sutskever, 2020) and retrieval-augmented proving (Yang et al., 2024b), the field has advanced through tree-search methods such as HyperTree Proof Search (Lample et al., 2022) and, more recently, large-scale neural provers including DeepSeek-Prover (Xin et al., 2024), Kimina-Prover (Wang et al., 2025), BFS-Prover (Xin et al., 2025a), and Goedel-Prover (Lin et al., 2025). Among these, Goedel-Prover is particularly noteworthy for its curriculum-driven synthetic data pipeline, which iteratively generates and verifies proof data to bootstrap prover capability.

However, the success of these systems does not directly transfer to code verification. In mathematics, a well-curated library such as `Mathlib` (mathlib Community, 2020) provides a dense, reusable inventory of lemmas, and the space of core concepts is comparatively concentrated. Code verification, by contrast, lacks a comparable lemma ecosystem: each program introduces its own function definitions, data-type invariants, and control-flow structures, effectively defining a fresh set of domain-specific concepts per task. This *concept proliferation* renders the proof distribution far more heterogeneous and precludes reliance on a fixed library of reusable lemmas, creating a substantial generalization gap for methods trained primarily on mathematical corpora.

Automated code verification with LLMs. Traditional program verification relies on SMT-based auto-active tools such as Dafny (Leino, 2010) and Verus (Lattuada et al., 2023), where correctness is established by discharging verification conditions through automated solvers. Recent efforts have explored using LLMs to assist these workflows, for example by generating loop invariants or annotations (Kamath et al., 2023; Pei et al., 2023). In the Lean ecosystem, Baldur (First et al., 2023) demonstrated whole-proof generation for software verification, and Rango (Thompson et al., 2024) introduced adaptive retrieval-augmented proving. However, these approaches either operate in a flat, single-pass generation mode or rely on tactic-level tree search without explicit goal decomposition. Our framework differs by introducing a planning layer that decomposes verification goals into structurally simpler sub-lemmas before invoking tactic-level proving, enabling the system to tackle problems whose complexity exceeds what flat generation can handle.

Hierarchical proof search and reinforcement learning. Decomposing complex goals into simpler sub-problems has a long history in automated reasoning (Li et al., 2024). In neural theorem proving, Draft-Sketch-Prove (Jiang et al., 2022) generates informal proof sketches before formalizing them, and subgoal-based approaches (Achim et al., 2025; Ren et al., 2025) have explored decomposition for mathematical theorems. AlphaProof (Hubert et al., 2025) achieved IMO-gold-level performance by combining reinforcement learning with a planning-and-proving paradigm. The most closely related prior system, ProD-RL (Dong et al., 2024), trains an Isabelle/AFP prover to propose and prove intermediate lemmas, rewarding correct and novel lemmas discovered during training. Our setting

and mechanism differ: we target Lean 4 code verification and use a continuous score that combines proof reconstruction, quickcheck filtering, and operator-footprint reduction, with the same score also ranking decompositions at inference time. DreamCoder (Ellis et al., 2020) instead learns reusable symbolic library abstractions for inductive program synthesis and does not decompose proof obligations through an interactive theorem prover. Our work adapts this paradigm to code verification, where useful decompositions differ fundamentally: intermediate assertions must capture semantic program properties (such as inductive invariants and termination measures) rather than syntactic refinements of the goal statement. On the reinforcement learning side, prior work has applied expert iteration (Polu & Sutskever, 2020), off-policy RL with tree search (Xin et al., 2025b), and process reward models (Lightman et al., 2024) to improve proof search. A persistent challenge is reward sparsity: binary proof success/failure signals provide limited gradient information. Our hybrid RL approach addresses this by introducing a continuous decomposition score as a dense reward for the planning stage, while stabilizing the proving stage through supervised replay.

C Verification Paradigms and Why Lean

Modern program verification predominantly relies on SMT-based "auto-active" verifiers such as Dafny (Leino, 2010) and Verus (Lattuada et al., 2023). These tools translate high-level code and annotations into verification conditions (VCs) discharged by automated solvers such as CVC5 (Barbosa et al., 2022) or Z3 (De Moura & Bjørner, 2008). While highly effective for systems engineering, the reasoning process is largely mediated by external SMT solvers, which act as black boxes from the LLM's perspective. When a VC fails, the solver typically returns a generic counterexample or timeout rather than a structured diagnostic, making it difficult for an LLM agent to identify the root cause and iteratively refine its annotations.

We adopt Lean 4 (Moura & Ullrich, 2021) as our verification platform for LLM-driven proof automation. From a foundational standpoint, Lean offers two structural advantages well-suited to LLM-driven workflows: (i) *Small trusted kernel.* Lean relies on a minimal logical kernel as its sole verification authority, so even when an LLM produces unpredictable or creative proof attempts, correctness is guaranteed by a small, well-audited checker rather than a complex solver stack. (ii) *Rich library ecosystem.* Lean's dependent type theory supports higher-order and richly structured specifications, and libraries such as `Mathlib` (mathlib Community, 2020) and `CSLib` (Barrett et al., 2026) provide an extensive knowledge base of verified lemmas and tactics that an LLM can directly invoke, facilitating the encoding of properties that extend beyond first-order SMT reasoning.

More importantly, Lean exposes the entire proof construction process to the LLM agent. Unlike opaque VC failures in SMT-based systems, Lean provides explicit goal states, local hypotheses, and fine-grained compiler diagnostics at each proof step. This transparency enables the LLM to plan proof strategies by inspecting open goals, execute tactics with immediate feedback, and debug failed attempts using structured error messages. Such an interactive loop is particularly suitable for iterative, model-driven verification.

Additionally, recent cross-system evaluations enable a controlled outcome comparison. The `AlgoVeri` benchmark (Zhao et al., 2026) translates the same set of algorithmic problems into Dafny, Verus, and Lean under aligned functional contracts, enabling a controlled comparison across verification paradigms. On its 77 aligned-contract tasks, the published Gemini-3-Flash evaluation reports success rates of 40.3% for Dafny, 24.7% for Verus, and 7.8% for Lean, whereas our Lean framework reaches 62.3% on the same task set. This controls the functional contracts but not the end-to-end inference compute, so the 22.0-point difference from Dafny should be interpreted as an outcome comparison rather than a compute-matched systems comparison. The result nevertheless shows that iterative proof interaction can make Lean competitive despite forgoing the raw automation of SMT solvers.

D Key Challenges in Code Verification

Despite recent advances in LLM-based theorem proving, automated code verification remains fundamentally harder than its mathematical counterpart. State-of-the-art planning-and-proving methods have achieved remarkable success in formal mathematics, reaching competition- and IMO-level performance (Hubert et al., 2025), yet their efficacy does not transfer to code verification. We identify three key challenges that explain this gap.

Ungrounded decomposition. Complex verification goals are seldom provable in a single pass; the prover must carefully plan its proof strategy and decompose the goal into auxiliary lemmas whose proofs collectively entail the top-level statement. In formal mathematics, LLMs acquire strong planning ability from extensive pretraining over textbooks, competition solutions, and online discussions, enabling them to propose meaningful decompositions even for challenging problems (Hubert et al., 2025; Chen et al., 2025). However, virtually no natural-language corpus describes how a program's specification is proved; reasoning about program correctness has no comparable textual footprint in LLM pretraining data. Without this grounding, frontier models frequently produce subgoals that are semantically invalid or no simpler than the original objective.

Concept proliferation. Even given a correct decomposition, proving each subgoal remains difficult because each program introduces a fresh conceptual universe. In formal mathematics, the relevant objects (real numbers, groups, combinatorial structures) form a relatively closed vocabulary supported by shared libraries such as Mathlib, and proofs can reuse a rich collection of existing lemmas. In contrast, every function definition in a program brings new computational behavior, control flow patterns, and data-structure invariants whose properties must be established from scratch without prior library support (Pei et al., 2023; Kamath et al., 2023). For example, the postcondition in Figure 1 relies on `filterlist`, a helper function defined within the program itself; no existing library contains lemmas about its behavior, so all necessary properties must be proved from scratch during verification.

Tactic distribution shift. Beyond the conceptual gap, the tactics needed to discharge code verification subgoals differ markedly from those prevalent in formal mathematics. Mathematical proofs rely heavily on algebraic normalization tactics such as `linarith`, `polyrith`, and `gcongr` over continuous domains. Code verification, by contrast, operates over discrete and recursive domains (e.g., machine integers, bit-vectors, lists, and graphs), requiring specialized automation such as `grind`, domain-specific decision procedures, and explicit recursive reasoning. For instance, proving properties about `filterlist` in our running example requires list induction and element-membership reasoning, handled by induction, simplification, or `grind` rather than the algebraic tactics that dominate mathematical proofs. Models trained primarily on mathematical corpora often fail to invoke these program-specific tools correctly, struggling to close even well-formulated subgoals.

The compound search challenge. Together, these three challenges create a compound search problem with an exceptionally fragile search space: a well-crafted set of subgoals can collapse a complex verification condition into tractable obligations, while a slightly misspecified lemma renders the entire proof intractable. This sensitivity motivates our approach, which trains a domain-specific decomposer guided by a principled scoring mechanism to identify decompositions that are both semantically valid and tactically provable.

E Complete Proof for the Running Example

Below is the complete machine-checked proof for `FindSingleNumber_spec` (the running example from Figure 1), as synthesized by our framework. The implementation shown here is the full version used in the actual proof; Figure 1 presents an equivalent but abbreviated form for readability. The proof decomposes the top-level theorem into six auxiliary lemmas covering list filtering, element counting, uniqueness, and the recursive scanner, totaling over 130 lines of proof code. Each block below corresponds to one lemma or the final theorem.

```
-- Precondition and postcondition (specification)
def FindSingleNumber_precond (nums : List Int) : Prop :=
  let numsCount := nums.map (fun x => nums.count x)
  numsCount.all (fun c => c = 1 ∨ c = 2)
    ∧ numsCount.count 1 = 1

def FindSingleNumber_postcond (nums : List Int)
    (result : Int)
    (h : FindSingleNumber_precond nums) : Prop :=
  nums.length > 0
    ∧ (filterlist result nums).length = 1
    ∧ ∀ (x : Int), x ∈ nums →
        x = result ∨ (filterlist x nums).length = 2
```

```
-- Helper function used in the specification
def filterlist (x : Int) (nums : List Int) : List Int :=
  let rec aux (lst : List Int) : List Int :=
    match lst with
    | []      => []
    | y :: ys => if y = x then y :: aux ys else aux ys
  aux nums

-- Implementation: find the unique element that appears exactly once
def FindSingleNumber (nums : List Int)
    (h_precond : FindSingleNumber_precond nums) : Int :=
  let rec findUnique (remaining : List Int) : Int :=
    match remaining with
    | [] => 0
    | x :: xs =>
      if (filterlist x nums).length = 1 then x
      else findUnique xs
  findUnique nums
```

```
-- Lemma 1: filterlist length equals List.count
lemma length_filterlist_eq_count (x : Int) (nums : List Int) :
    (filterlist x nums).length = nums.count x := by
  classical
  have aux_eq : ∀ lst : List Int,
      filterlist.aux x lst = lst.filter (fun y => y = x) := by
    intro lst; induction lst with
    | nil => simp [filterlist.aux]
    | cons y ys ih =>
        by_cases h : y = x <;> simp [filterlist.aux, ih, h]
  have eq_filter :
      filterlist x nums = nums.filter (fun y => y = x) :=
    by simp [filterlist, aux_eq]
  simpa [eq_filter] using
    (List.count_eq_length_filter (l := nums) (a := x)).symm
```

```
-- Lemma 2: the precondition guarantees a unique element exists
lemma exists_count_eq_one (nums : List Int)
    (h_precond : FindSingleNumber_precond nums) :
    ∃ x ∈ nums, nums.count x = 1 := by
  classical
  unfold FindSingleNumber_precond at h_precond
  rcases h_precond with ⟨_hall, huniq⟩
  have hmem : 1 ∈ (nums.map (fun x => nums.count x)) := by
    by_contra hnot
    have : (nums.map (fun x => nums.count x)).count 1 = 0 :=
      by simp [List.count_eq_zero, hnot]
    exact Nat.one_ne_zero (by simpa [this] using huniq)
  rcases List.mem_map.mp hmem with ⟨x, hx, hxcount⟩
```

```
  exact ⟨x, hx, by simpa using hxcount⟩
```

```
-- Lemma 3: the recursive scanner returns a unique element
lemma findUnique_returns_length_one (nums : List Int) :
    ∀ remaining : List Int,
      (∃ u ∈ remaining, (filterlist u nums).length = 1) →
        (filterlist (FindSingleNumber.findUnique
          (nums := nums) remaining) nums).length = 1 := by
  classical
  intro remaining; induction remaining with
  | nil => intro hex; rcases hex with ⟨u, hu, _⟩; cases hu
  | cons x xs ih =>
      intro hex; simp [FindSingleNumber.findUnique]
      by_cases hx1 : (filterlist x nums).length = 1
      · simp [hx1]
      · have hex' :
            ∃ u ∈ xs, (filterlist u nums).length = 1 := by
          rcases hex with ⟨u, hu_mem, hu_len⟩; simp at hu_mem
          cases hu_mem with
          | inl hu_eq =>
              subst hu_eq; exact False.elim (hx1 hu_len)
          | inr hu_mem_xs =>
              exact ⟨u, hu_mem_xs, hu_len⟩
        simpa [hx1] using ih hex'
```

```
-- Lemma 4: the algorithm returns a value with count = 1
lemma FindSingleNumber_result_characterization
    (nums : List Int)
    (h_precond : FindSingleNumber_precond nums) :
    nums.count (FindSingleNumber nums h_precond) = 1 := by
  classical
  rcases exists_count_eq_one nums h_precond
    with ⟨u, hu_mem, hu_count⟩
  have hu_len : (filterlist u nums).length = 1 :=
    by simp [length_filterlist_eq_count, hu_count]
  have hlen := by simpa [FindSingleNumber] using
    findUnique_returns_length_one nums nums
      ⟨u, hu_mem, hu_len⟩
  simpa [length_filterlist_eq_count] using hlen
```

```
-- Lemma 5: uniqueness of the element with count = 1
lemma precond_unique_count_one (nums : List Int)
    (h_precond : FindSingleNumber_precond nums)
    {a b : Int} (ha : a ∈ nums) (hb : b ∈ nums)
    (hca : nums.count a = 1) (hcb : nums.count b = 1) :
    a = b := by
  classical
  unfold FindSingleNumber_precond at h_precond
  rcases h_precond with ⟨_hall, huniq⟩
  by_contra hne
  let p : Int → Prop := fun x => nums.count x = 1
  have haF : a ∈ nums.filter p :=
    List.mem_filter.2 ⟨ha, by simp [p, hca]⟩
  have hbF : b ∈ nums.filter p :=
    List.mem_filter.2 ⟨hb, by simp [p, hcb]⟩
  have two_le {ff : Type} {l : List ff} {a b : ff}
      (ha : a ∈ l) (hb : b ∈ l) (hne : a ≠ b) :
      2 ≤ l.length := by
    induction l with
    | nil => cases ha
    | cons c tl ih =>
```

```
      have ha' : a = c ∨ a ∈ tl := by simpa using ha
      have hb' : b = c ∨ b ∈ tl := by simpa using hb
      cases ha' with
      | inl hac => cases hb' with
        | inl hbc =>
            exact (hne (by simpa [hac] using hbc.symm)).elim
        | inr hb_tl =>
            exact Nat.succ_le_succ
              (Nat.succ_le_of_lt (List.length_pos_of_mem hb_tl))
      | inr ha_tl => cases hb' with
        | inl hbc =>
            exact Nat.succ_le_succ
              (Nat.succ_le_of_lt (List.length_pos_of_mem ha_tl))
        | inr hb_tl =>
            exact le_trans (ih ha_tl hb_tl) (Nat.le_succ tl.length)
  have hlen_ge : 2 ≤ (nums.filter p).length :=
    two_le haF hbF hne
  have count_map_eq_aux : ∀ l : List Int,
      (l.map (fun x => nums.count x)).count 1 =
        (l.filter (fun a => nums.count a = 1)).length := by
    intro l; induction l with
    | nil => simp
    | cons a l ih =>
        by_cases h : nums.count a = 1 <;> simp [p, h, ih]
  have hlen_eq_one : (nums.filter p).length = 1 := by
    simp only [p]
    exact (count_map_eq_aux nums).symm.trans huniq
  exact Nat.not_succ_le_self 1
    (by simpa [hlen_eq_one] using hlen_ge)
```

```
-- Lemma 6: non-unique elements have count = 2
lemma count_eq_two_of_ne_of_count_eq_one
    (nums : List Int)
    (h_precond : FindSingleNumber_precond nums)
    (r x : Int) (hx : x ∈ nums)
    (hcr : nums.count r = 1) (hxr : x ≠ r) :
    nums.count x = 2 := by
  classical
  unfold FindSingleNumber_precond at h_precond
  rcases h_precond with ⟨hall, huniq⟩
  have precond_counts (x' : Int) (hx' : x' ∈ nums) :
      nums.count x' = 1 ∨ nums.count x' = 2 := by
    have hall' : ∀ n ∈ (nums.map fun y => nums.count y),
        (n = 1 ∨ n = 2) := by
      simpa [List.all_eq_true] using hall
    exact hall' (nums.count x')
      (List.mem_map.mpr ⟨x', hx', rfl⟩)
  have h_precond' : FindSingleNumber_precond nums := by
    unfold FindSingleNumber_precond; exact ⟨hall, huniq⟩
  have mem_of_one (a : Int) (h : nums.count a = 1) :
      a ∈ nums := by
    by_contra hmem
    have : nums.count a = 0 := by
      simp [List.count_eq_zero, hmem]
    exact Nat.zero_ne_one (by simpa [this] using h)
  cases precond_counts x hx with
  | inl h1 =>
      exact (hxr (precond_unique_count_one nums h_precond'
        hx (mem_of_one r hcr) h1 hcr)).elim
  | inr h2 => exact h2
```

```
-- Main theorem: combining all lemmas
theorem FindSingleNumber_spec_satisfied
    (nums : List Int)
    (h_precond : FindSingleNumber_precond nums) :
    FindSingleNumber_postcond nums
      (FindSingleNumber nums h_precond) h_precond := by
  classical
  constructor
  · by_contra h0
    have hnil : nums = [] := by
      simpa using List.length_eq_zero_iff.mp
        (Nat.eq_zero_of_not_pos h0)
    subst hnil
    simp [FindSingleNumber_precond] at h_precond
  constructor
  · simpa [length_filterlist_eq_count] using
      FindSingleNumber_result_characterization nums h_precond
  · intro x hx
    by_cases h : x = FindSingleNumber nums h_precond
    · exact Or.inl h
    · have hcount :=
        count_eq_two_of_ne_of_count_eq_one nums h_precond
          (FindSingleNumber nums h_precond) x hx
          (FindSingleNumber_result_characterization
            nums h_precond) h
      exact Or.inr
        (by simp [length_filterlist_eq_count, hcount])
```

F Algorithm Pseudocode

We provide the full pseudocode for the two core procedures of our framework. Algorithm 1 details the hybrid reinforcement learning loop that jointly optimizes decomposition via GRPO and completion via supervised replay, while progressively expanding the problem set with newly decomposed lemmas. Algorithm 2 describes the two-stage hierarchical inference pipeline, where the policy first recursively decomposes the verification goal and then attempts to prove each remaining leaf lemma with iterative refinement.

Algorithm 1 Hybrid RL with Online Lemma Collection

Require: Initial problem set $\mathcal{P}_0$, policy π, loss coefficient λ

for iterations $t = 1, 2, \ldots$ **do**
  Sample problem $G \sim \mathcal{P}_{t-1}$.
  Roll out π to produce decomposed lemmas $\mathcal{L}_t := \{L_1, \ldots, L_k\}$.
  Compute reward $S = r(L_1, \ldots, L_k; G) \cdot v(L_1, \ldots, L_k; G)$ and GRPO objective $\mathcal{J}_{\text{GRPO}}$.
  Attempt completion of $\mathcal{L}_t$: first roll out π; if π fails within m attempts, fall back to an off-the-shelf model to produce $\mathcal{T}_t := \{\tau_1, \ldots, \tau_k\}$.
  Compute SFT objective $\mathcal{J}_{\text{SFT}}$ on completion trajectories $(\mathcal{L}_t, \mathcal{T}_t)$.
  Update π with hybrid objective $\mathcal{J} = \mathcal{J}_{\text{GRPO}} + \lambda \cdot \mathcal{J}_{\text{SFT}}$.
  Augment problem set: $\mathcal{P}_t \leftarrow \mathcal{P}_{t-1} \cup \mathcal{L}_t$.
end for

G Implementation Details

Custom Lean Tactics. We implement two custom Lean tactics used by the decomposition scoring mechanism. `operatorcount` traverses the goal's abstract syntax tree and returns the total number of logical operators (connectives, quantifiers) and domain-specific program operators (arithmetic, bitwise, data-structure constructors), excluding variable references and type annotations. `quickcheck` extends the `Plausible` library (Lean Prover Community, 2024): given a universally quantified lemma, it randomly samples concrete inputs (up to

Algorithm 2 Hierarchical Inference (Single Run)

Require: Theorem G, policy π
// Stage 1: Lemma Decomposition
Initialize open goal set $\mathcal{O} \leftarrow \{G\}$.
for decomposition iterations $i = 1, \ldots, 128$ **do**
  Select the goal $g \in \mathcal{O}$ with the largest operator footprint.
  Roll out π to decompose g into sub-lemmas $\{L_1, \ldots, L_k\}$.
  Verify proof reconstruction and quickcheck each L_i; if either check fails, discard and retry.
  If the update would exceed 32 open goals, stop decomposition.
  Update $\mathcal{O}$: remove g, add $\{L_1, \ldots, L_k\}$; compute operator footprints.
end for
// Stage 2: Lemma Completion
Collect remaining leaf lemmas $\mathcal{L} = \{L_1, \ldots, L_m\}$ from $\mathcal{O}$.
for completion iterations $j = 1, \ldots, 128$ **do**
  Roll out π to generate candidate proofs for each unclosed $L_i \in \mathcal{L}$.
  Refine proofs based on Lean compiler feedback.
  if all lemmas closed **then return** success.
end for

return failure.

1000 trials) and evaluates the lemma via native execution, reporting a counterexample if one is found. Both tactics are registered as Lean meta-programs and can be invoked at any proof state during decomposition verification.

Data Curation. We collect 5K unique problems from LeetCode (Xia et al., 2025) and OpenCodeInstruct (Ahmad et al., 2025). Each problem is automatically formalized into Lean using GPT-5.2. The formalizations undergo iterative refinement until they pass both syntactic validation and quickcheck filtering, ensuring empirical consistency. To prevent data contamination, we remove all problems whose title or description has an exact match with any problem in the Verina, Clever, or AlgoVeri evaluation benchmarks. We additionally audit the resulting split using longest-shared-substring and embedding-similarity checks on boilerplate-stripped Lean implementations; Appendix H.2 reports the thresholds and flagged pairs.

Trajectory Filtering. We apply different filtering criteria for the two trajectory types. For decomposition trajectories, we retain only those that are constructively justified (the proposed lemmas provably entail the parent theorem) and achieve a positive structural-reduction score ($S > 0$). For completion trajectories, we retain only those whose final proof is accepted by the Lean kernel.

Supervised Fine-Tuning. We fine-tune Qwen3-8B (Yang et al., 2025) using LLaMA-Factory (Zheng et al., 2024) for three epochs on a mixture of decomposition and proof completion trajectories. Training uses a learning rate of 1×10^{-5} with a cosine schedule and a warmup ratio of 0.1. The maximum context length is 10,240 tokens with sequence packing enabled. We use a batch size of 128 with sequence packing.

Reinforcement Learning. We apply GRPO (Shao et al., 2024) using the `verl` framework (Sheng et al., 2024) for 100 training steps across 16 GPUs. The learning rate is 5×10^{-6} with cosine decay, with a clip ratio of [0.2, 0.28] and a sampling temperature of 0.9. We use a batch size of 64 prompts with n=8 parallel generations per prompt and a mini-batch size of 256 for policy update. Rewards are computed by verifying proof attempts and decomposition quality as described. To ensure informative gradient signals, we filter out rollout groups whose mean reward is 0 or 1. We additionally apply an auxiliary supervised

Decomposition Prompt Template

System: You are an expert in Lean 4 theorem proving. Your task is to decompose a theorem into smaller, provable lemmas.
Think step by step. Before writing code, first analyze the theorem structure and explain your decomposition strategy in a `<reasoning>` block.
Decomposition Strategies:
- **Structural Decomposition** — Match goal structure: split $P \wedge Q$ into separate lemmas; prove one side of $P \vee Q$; find witness for $\exists x, P(x)$.
- **Induction Decomposition** — For recursive structures: base case + inductive step as separate lemmas.
- **Rewrite Chain** — For equational goals $A = D$: break into $A = B$, $B = C$, $C = D$.
- **Case Analysis** — Split by `Decidable`, `match`, or `if-then-else`; each branch becomes a separate lemma.

Output Requirements: (1) Wrap analysis in `<reasoning>...</reasoning>` tags before any code; (2) Helper lemmas use `lemma` keyword with `sorry` body; (3) Main theorem uses `theorem` keyword and proves using the lemmas (no `sorry`).

User: Decompose the theorem {`theorem_name`}.
{`formal_problem`}

Figure 6: Prompt template for the decomposition stage. The model receives the target theorem name and the full Lean formalization, and produces a reasoning trace followed by helper lemmas and a proof of the main theorem.

Completion Prompt Template

System: Lean 4 proof assistant. Complete the proof by filling in the `sorry` placeholder with valid tactics.
Important: Output reasoning in a `reasoning` block before any diff blocks.
Recommended tactics (try in order): `grind` (powerful automation), `aesop` (general automation), `simp_all` (aggressive simplification), `omega` (linear arithmetic), `by_cases` (case split), `induction` (induction with auto finish).
Output format: SEARCH/REPLACE diff blocks.

User: Complete the current proof goal.
{`code`}
Goal (Line {`line`}): {`goal`}
After the reasoning block, output diff blocks only.

Figure 7: Prompt template for the completion stage. The model receives the current code, the target goal state, and responds with SEARCH/REPLACE diffs that fill in proof bodies.

fine-tuning loss on proof completion trajectories with coefficient λ=0.08 and refresh the online proof-state buffer at a sampling ratio of $\frac{1}{4}$ per step.

Prompting Format. For decomposition, the prompt presents the current Lean file with the target theorem highlighted, and instructs the model to produce a `<reasoning>` block followed by helper lemmas (each with `sorry` placeholders) and a main theorem whose proof invokes these lemmas without `sorry`. For completion, the prompt shows the current goal state and compilation status, and the model responds with a SEARCH/REPLACE diff that modifies only the proof body while preserving theorem signatures and imports. We use the same prompting format for both training and inference. The full prompt templates are shown in Figure 6 and Figure 7.

Inference Configuration. Each problem is allocated a wall-clock budget of 30 minutes. The decomposition stage runs up to 128 iterations with a maximum of 32 lemmas per problem. The completion stage runs up to 128 iterations per lemma, with a two-phase strategy: (1) attempt tactic-based proving when the code compiles, and (2) fix compilation

errors if present. We deploy a distributed Lean verification service built on Ray (Moritz et al., 2018), which manages a pool of Lean worker processes across multiple nodes and supports up to 512 concurrent verification requests with a 300-second timeout per check.

H Additional Results

H.1 Off-the-Shelf Open-Model Instantiation

Table 4: Verina results (%) when instantiating the framework with off-the-shelf models without fine-tuning.

Decomposition	Completion	Verina
—	GPT-OSS-120B	20.1
—	Gemini-3-Flash	19.6
GPT-OSS-120B	GPT-OSS-120B	44.9
GPT-5.2-Pro	Gemini-3-Flash	54.4

Using GPT-OSS-120B for both stages raises its success rate from 20.1% in whole-proof generation to 44.9% with hierarchical search. This result shows that the structural benefit of decomposition is not restricted to the frontier models used for data collection; it does not, however, substitute for a fully frontier-free training run.

H.2 Training–Evaluation Decontamination Audits

We supplement title/description exact matching with two audits on boilerplate-stripped Lean implementations. First, following the ExactSubstr-style audit of Lee et al. (2022), we compare each benchmark implementation with all 4,277 retained training implementations.

Table 5: Benchmark tasks whose implementation shares a contiguous substring with a training implementation.

Threshold	Verina	Clever	AlgoVeri	Total
$\geq$ 50 chars	7	5	1	13/427
$\geq$ 100 chars	3	0	0	3/427
$\geq$ 200 chars	0	0	0	0/427

The Clever and AlgoVeri hits at 50 characters are generic Lean fragments rather than shared problem content; neither benchmark has a hit at 100 characters, and no benchmark task has a hit at 200 characters. Second, we embed each stripped reference implementation with `text-embedding-3-large` and flag nearest training neighbors with cosine similarity at least 0.95. Only 2/427 tasks (0.47%), both from Verina, are flagged: `verina_advanced_42/maxProfit` (0.997) and `verina_advanced_18/isArmstrong` (0.964). Manual inspection finds materially different specifications or implementations in both pairs, so neither admits direct proof transfer. Together with title/description filtering, these audits find no evidence of proof-level leakage; the residual Verina similarity is consistent with independently formalizing tasks from the same LeetCode source.

H.3 Decomposition Score Beyond the Evaluation Benchmarks

We evaluate the operator-footprint reduction ratio r on 22 CSLib and 10 Mathlib theorems with human-written lemma decompositions.

Table 6: Behavior of the operator-footprint reduction ratio on human-written decompositions. The ratio is informative primarily for Hoare-style program-correctness statements, its intended setting.

Theorem class	Cases	$r > 0$	Representative examples (reduction ratio)
Hoare-style program correctness (CSLib)	14	9	`otp_ciphertextDist_eq_uniform` (0.872), `mergeSort_sorted` (0.502), `toNAFinAcc_language_eq` (0.492), Shamir privacy (0.154), and `pred_correct` for SKI (0.112)
Type-safety theorems (CSLib)	3	0	STLC/Fsub preservation and progress statements
Structural and categorical theorems (CSLib)	5	0	LTS composition, Turing-machine composition, ω-regular complement, CLL η-expansion, and infinite Ramsey
Pure mathematics (Mathlib)	10	0	Lagrange, Beatty, Hensel, Siegel, Kronecker, Shortlex, Lucas–Lehmer, and related theorems

Nine of the 14 Hoare-style cases receive a positive reduction ratio, while all abstract type-safety, structural, categorical, and pure-mathematics cases receive zero. This behavior matches the score's intended asymmetry: program postconditions often unfold into many program operators, whereas useful helper lemmas isolate simpler sub-properties. The result also delineates the metric's scope—operator-footprint reduction is not a general-purpose measure of mathematical decomposition quality.

H.4 Inference Budgets and Comparison Scope

Our system allocates 30 minutes per problem, runs up to 128 decomposition iterations with at most 32 open lemmas, and then runs up to 128 completion iterations per lemma; pass@k launches k independent runs. The reported baselines use their default decoding configurations at pass@128, and the baseline scaling curves in Figure 9 extend BFS-Prover-V2 and Göedel-Prover-V2 to pass@1024. Because closed models expose token/API accounting while open provers use heterogeneous model sizes and decoding implementations, these evaluations are not compute-matched. Accordingly, our comparisons concern verified success under the stated budgets, not equal-cost efficiency or an effect attributable to parameter count.

H.5 Proof Statistics

Table 7: Statistics of proved problems: mean and standard deviation of lemma count and proof length (lines of code).

Statistics	Verina	Clever	AlgoVeri	Total
Lemma count (mean)	17.02	12.13	8.48	14.38
Lemma count (std)	11.79	10.57	10.97	11.97
Proof length (mean)	167.11	137.78	129.87	154.28
Proof length (std)	108.50	97.79	83.03	103.11

Table 7 reports the mean and standard deviation of lemma count and proof length (in lines of code) for problems successfully proved by our framework across the three benchmarks.

H.6 Residual Operator-Footprint Ratio

Figure 8 shows how the residual operator-footprint ratio evolves as iterations progress under different pass@k budgets. The ratio decreases steadily, indicating that the framework identifies increasingly effective goal simplifications with additional compute.

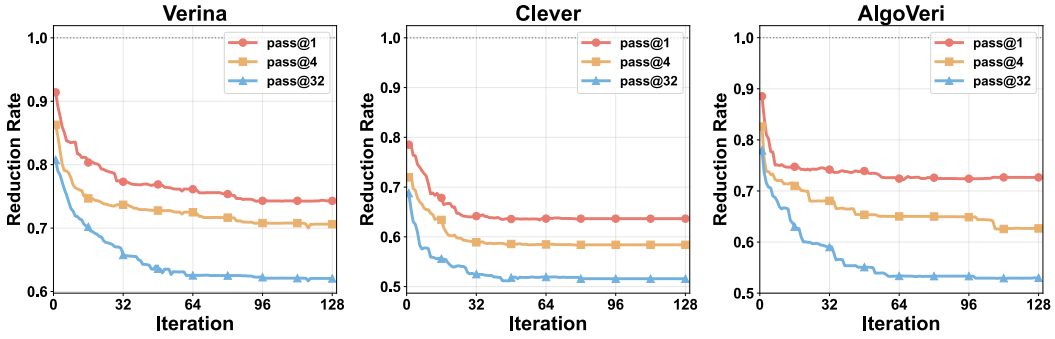

Figure 8: Residual operator-footprint ratio $\bar{d}/d(G)$ vs. iterations across three benchmarks. Lower values indicate more aggressive goal simplification and therefore a higher reduction ratio r.

H.7 Baseline Inference Scaling Curves

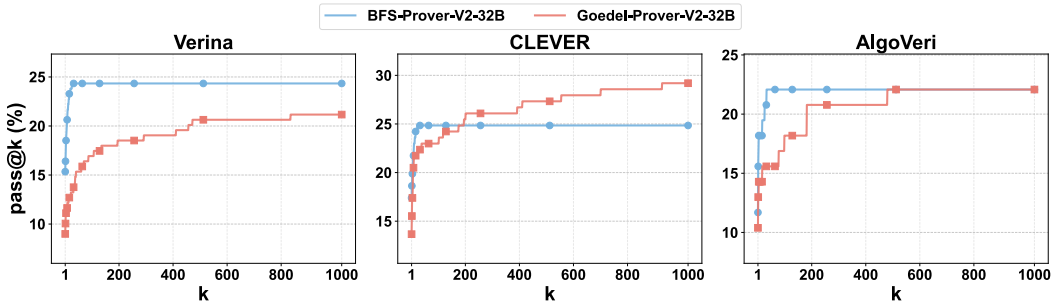

Figure 9: Inference scaling curves (pass@k) of two baseline verifiers, BFS-Prover-V2-32B and Göedel-Prover-V2-32B, on the three evaluation benchmarks. The x-axis denotes the number of sampled proof attempts k, and the y-axis shows the percentage of problems solved.

To contextualize the inference-time scaling behavior of our framework, we examine the pass@k curves of the two strongest baseline verifiers. As shown in Figure 9, both BFS-Prover-V2-32B and Göedel-Prover-V2-32B exhibit diminishing returns as k increases: performance plateaus well before k=1024 on all three benchmarks, suggesting that simply scaling the number of independent proof attempts provides limited gains for whole-proof generation approaches. In contrast, our hierarchical framework continues to improve with additional compute (Section 4.3), highlighting the advantage of structured search over brute-force sampling.

H.8 Decomposition Score Correlation on Training Data

In Section 4.4, we show that the unnormalized decomposition score is a strong inverse predictor of downstream provability on Verina (AUROC = 0.903). We repeat this analysis on the training data to determine whether the association is unique to that evaluation set. As shown in Figure 10, problems with lower raw scores are also more likely to be proved on the training distribution. This in-distribution result shows that the association is not unique to Verina, but it does not by itself establish out-of-distribution generalization.

H.9 Effect of Training on Decomposition Quality

Across Verina, Clever, and AlgoVeri, the mean raw score falls from 282 to 185, 437 to 329, and 271 to 196, respectively; lower raw scores correspond to structurally simpler decompositions and higher normalized S.

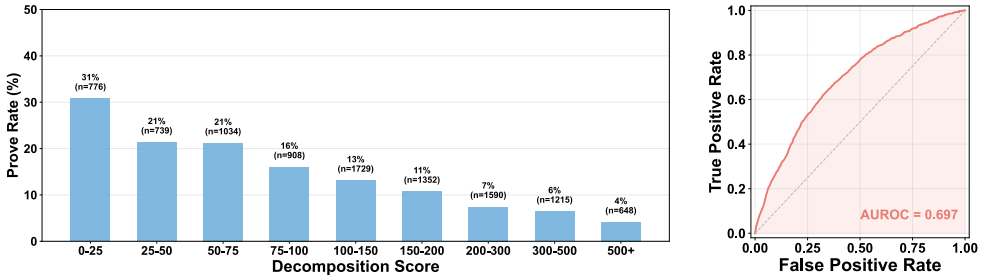

Figure 10: Unnormalized decomposition score vs. prove rate on training data. As on Verina, lower raw scores are associated with higher prove rates.

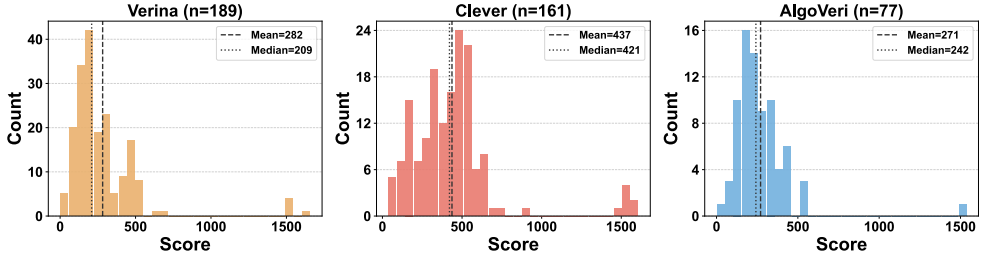

Figure 11: Unnormalized decomposition-score distribution before training across benchmarks. Lower values are better.

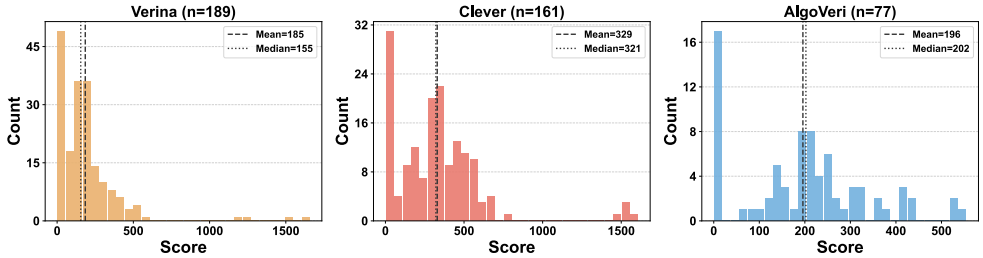

Figure 12: Unnormalized decomposition-score distribution after training across benchmarks. Lower values are better.